

Cognizant-upGrad Training Program

uGE_Dotnet FSD with Python

Weekly Submission Document

Name: Sunkara Srilatha

Email: srisrilatha9999@gmail.com

Batch Name: B1_Python

Course: uGE_Dotnet FSD (Python)

Trainer: Kunal

WEEK 9

1. Smart Expense Tracker (CLI Based)

Problem Statement

You need to create a command-line application that helps a user track their daily expenses.

```
import os
```

```
FILE_NAME = "expenses.txt"
```

```
# Add Expense
```

```
def add_expense():
```

```
    amount = input("Enter amount: ")
```

```
    category = input("Enter category: ")
```

```
    date = input("Enter date (YYYY-MM-DD): ")
```

```
    with open(FILE_NAME, "a") as file:
```

```
        file.write(f"{amount},{category},{date}\n")
```

```
    print("Expense added successfully!\n")
```

```
# View All Expenses
```

```
def view_expenses():
```

```
    if not os.path.exists(FILE_NAME):
```

```
print("No expenses found.\n")
```

```
return
```

```
with open(FILE_NAME, "r") as file:
```

```
    data = file.readlines()
```

```
    print("\nAll Expenses:")
```

```
    for line in data:
```

```
        amount, category, date = line.strip().split(",")
```

```
        print(f"Amount: {amount}, Category: {category}, Date: {date}")
```

```
    print()
```

```
# Filter by Category
```

```
def filter_category():
```

```
    cat = input("Enter category to filter: ")
```

```
if not os.path.exists(FILE_NAME):
```

```
    print("No data found.\n")
```

```
    return
```

```
with open(FILE_NAME, "r") as file:
```

```
    data = file.readlines()
```

```
    print(f"\nExpenses in {cat}:")
```

```
    for line in data:
```

```
        amount, category, date = line.strip().split(",")
```

```
        if category.lower() == cat.lower():
```

```
            print(f"Amount: {amount}, Date: {date}")
```

```
    print()
```

```
# Monthly Summary
```

```
def monthly_summary():

    month = input("Enter month (YYYY-MM): ")

    total = 0


    if not os.path.exists(FILE_NAME):

        print("No data found.\n")

        return


    with open(FILE_NAME, "r") as file:

        data = file.readlines()


        for line in data:

            amount, category, date = line.strip().split(",")

            if date.startswith(month):

                total += float(amount)


    print(f"Total expense for {month}: {total}\n")


# Main Menu

def main():

    while True:

        print("==== Expense Tracker =====")

        print("1. Add Expense")

        print("2. View All Expenses")

        print("3. Filter by Category")

        print("4. Monthly Summary")

        print("5. Exit")


        choice = input("Enter your choice: ")


        if choice == "1":
```

```

        add_expense()
    elif choice == "2":
        view_expenses()
    elif choice == "3":
        filter_category()
    elif choice == "4":
        monthly_summary()
    elif choice == "5":
        print("Exiting...")
        break
    else:
        print("Invalid choice\n")

if __name__ == "__main__":
    main()

```

2. Student Management System (OOP Based)

Problem Statement

Create a system to manage student records using classes.

```
import os
```

```
FILE_NAME = "students.txt"
```

```
# Student Class
```

```
class Student:
```

```
    def __init__(self, name, roll, marks):
```

```
        self.name = name
```

```
        self.roll = roll
```

```
        self.marks = marks
```

```
    def display(self):
```

```
        print(f"Name: {self.name}, Roll: {self.roll}, Marks: {self.marks}")
```

```
# Add Student
```

```
def add_student():
```

```
    name = input("Enter name: ")
```

```
    roll = input("Enter roll number: ")
```

```
    marks = input("Enter marks: ")
```

```
    with open(FILE_NAME, "a") as file:
```

```
        file.write(f"{name},{roll},{marks}\n")
```

```
    print("Student added successfully!\n")
```

```
# View Students
```

```
def view_students():
```

```
    if not os.path.exists(FILE_NAME):
```

```
        print("No data found.\n")
```

```
        return
```

```
    with open(FILE_NAME, "r") as file:
```

```
        print("\nAll Students:")
```

```
        for line in file:
```

```
            if line.strip() == "":
```

```
                continue
```

```
            parts = line.strip().split(",")
```

```
            if len(parts) != 3:
```

```
                continue
```

```
            name, roll, marks = parts
```

```
            student = Student(name, roll, marks)
```

```
        student.display()
    print()
```

```
# Search Student
```

```
def search_student():
```

```
    roll_no = input("Enter roll number to search: ")
```

```
    if not os.path.exists(FILE_NAME):
```

```
        print("No data found.\n")
```

```
    return
```

```
found = False
```

```
with open(FILE_NAME, "r") as file:
```

```
    for line in file:
```

```
        if line.strip() == "":
```

```
            continue
```

```
        name, roll, marks = line.strip().split(",")
```

```
        if roll == roll_no:
```

```
            print("Student Found:")
```

```
            print(f"Name: {name}, Marks: {marks}")
```

```
            found = True
```

```
if not found:
```

```
    print("Student not found.\n")
```

```
# Update Marks
```

```
def update_marks():
```

```
    roll_no = input("Enter roll number to update: ")
```

```
if not os.path.exists(FILE_NAME):
    print("No data found.\n")
    return

new_data = []
updated = False

with open(FILE_NAME, "r") as file:
    for line in file:
        if line.strip() == "":
            continue

        name, roll, marks = line.strip().split(",")

        if roll == roll_no:
            marks = input("Enter new marks: ")
            updated = True

        new_data.append(f"{name},{roll},{marks}\n")

with open(FILE_NAME, "w") as file:
    file.writelines(new_data)

if updated:
    print("Marks updated successfully!\n")
else:
    print("Student not found.\n")

# Delete Student
def delete_student():
    roll_no = input("Enter roll number to delete: ")
```

```
if not os.path.exists(FILE_NAME):
```

```
    print("No data found.\n")
```

```
    return
```

```
new_data = []
```

```
deleted = False
```

```
with open(FILE_NAME, "r") as file:
```

```
    for line in file:
```

```
        if line.strip() == "":
```

```
            continue
```

```
        name, roll, marks = line.strip().split(",")
```

```
        if roll != roll_no:
```

```
            new_data.append(f"{name},{roll},{marks}\n")
```

```
        else:
```

```
            deleted = True
```

```
with open(FILE_NAME, "w") as file:
```

```
    file.writelines(new_data)
```

```
if deleted:
```

```
    print("Student deleted successfully!\n")
```

```
else:
```

```
    print("Student not found.\n")
```

```
# Main Menu
```

```
def main():
```

```
    while True:
```



```
print("==== Student Management System ====")  
print("1. Add Student")  
print("2. View Students")  
print("3. Search Student")  
print("4. Update Marks")  
print("5. Delete Student")  
print("6. Exit")
```

```
choice = input("Enter your choice: ")
```

```
if choice == "1":  
    add_student()  
elif choice == "2":  
    view_students()  
elif choice == "3":  
    search_student()  
elif choice == "4":  
    update_marks()  
elif choice == "5":  
    delete_student()  
elif choice == "6":  
    print("Exiting...")  
    break  
else:  
    print("Invalid choice\n")
```

```
if __name__ == "__main__":  
    main()
```

3. Library Management System

Problem Statement

Build a system to manage books in a small library.

```
import os

FILE_NAME = "books.txt"

# Add Book
def add_book():
    name = input("Enter book name: ")
    author = input("Enter author name: ")

    with open(FILE_NAME, "a") as file:
        file.write(f"{name},{author},Available\n")

    print("Book added successfully!\n")

# View Available Books
def view_books():
    if not os.path.exists(FILE_NAME):
        print("No books found.\n")
        return

    print("\nAvailable Books:")
    with open(FILE_NAME, "r") as file:
        for line in file:
            if line.strip() == "":
                continue

            name, author, status = line.strip().split(",")

            if status == "Available":
                print(f"Book: {name}, Author: {author}")
    print()
```

```
# Issue Book
```

```
def issue_book():
```

```
    book_name = input("Enter book name to issue: ")
```

```
    user = input("Enter your name: ")
```

```
    if not os.path.exists(FILE_NAME):
```

```
        print("No books found.\n")
```

```
        return
```

```
    new_data = []
```

```
    found = False
```

```
    with open(FILE_NAME, "r") as file:
```

```
        for line in file:
```

```
            if line.strip() == "":
```

```
                continue
```

```
            name, author, status = line.strip().split(",")
```

```
            if name.lower() == book_name.lower() and status == "Available":
```

```
                status = f"Issued to {user}"
```

```
                found = True
```

```
            new_data.append(f"{name},{author},{status}\n")
```

```
    with open(FILE_NAME, "w") as file:
```

```
        file.writelines(new_data)
```

```
    if found:
```

```
        print("Book issued successfully!\n")
```

```
else:
```

```
    print("Book not available.\n")
```

```
# Return Book
```

```
def return_book():
```

```
    book_name = input("Enter book name to return: ")
```

```
    if not os.path.exists(FILE_NAME):
```

```
        print("No books found.\n")
```

```
        return
```

```
    new_data = []
```

```
    found = False
```

```
    with open(FILE_NAME, "r") as file:
```

```
        for line in file:
```

```
            if line.strip() == "":
```

```
                continue
```

```
            name, author, status = line.strip().split(",")
```

```
            if name.lower() == book_name.lower() and "Issued" in status:
```

```
                status = "Available"
```

```
                found = True
```

```
            new_data.append(f"{name},{author},{status}\n")
```

```
    with open(FILE_NAME, "w") as file:
```

```
        file.writelines(new_data)
```

```
    if found:
```

```

        print("Book returned successfully!\n")
    else:
        print("Book not found or already available.\n")

# Main Menu
def main():
    while True:
        print("==== Library Management System =====")
        print("1. Add Book")
        print("2. Issue Book")
        print("3. Return Book")
        print("4. View Available Books")
        print("5. Exit")

        choice = input("Enter your choice: ")

        if choice == "1":
            add_book()
        elif choice == "2":
            issue_book()
        elif choice == "3":
            return_book()
        elif choice == "4":
            view_books()
        elif choice == "5":
            print("Exiting...")
            break
        else:
            print("Invalid choice\n")

if __name__ == "__main__":

```

```
main()
```

4. Log Analyzer Tool

Problem Statement

You are given a log file containing system logs.
You need to analyze and extract useful information.

```
import re

from collections import Counter

def analyze_logs(file_path):
    with open(file_path, 'r') as file:
        logs = file.readlines()

    ip_list = []
    date_list = []
    error_lines = []

    ip_pattern = r'\b(?:[0-9]{1,3}\.){3}[0-9]{1,3}\b'
    date_pattern = r'\b\d{4}-\d{2}-\d{2}\b'

    for line in logs:
        ips = re.findall(ip_pattern, line)
        ip_list.extend(ips)

        dates = re.findall(date_pattern, line)
        date_list.extend(dates)

        if "ERROR" in line:
            error_lines.append(line.strip())

    total_errors = len(error_lines)
    ip_count = Counter(ip_list)
```

```

print("IP Addresses:", ip_list)
print("Dates:", date_list)
print("Errors:", error_lines)
print("Total Errors:", total_errors)

print("IP Count:")
for ip, count in ip_count.items():
    print(ip, "->", count)

with open("summary.txt", "w") as file:
    file.write("Total Errors: " + str(total_errors) + "\n")
    for ip, count in ip_count.items():
        file.write(ip + " -> " + str(count) + "\n")

```

```
analyze_logs("log.txt")
```

5. Task Automation Tool

Problem Statement

Create a tool that performs operations on files inside a folder.

```
import os
```

```

def list_files(path):
    files = os.listdir(path)
    print("\nFiles in folder:")
    for file in files:
        print(file)

```

```

def rename_files(path):
    files = os.listdir(path)
    count = 1

```

```
for file in files:
```

```
    old_path = os.path.join(path, file)
```

```
    if os.path.isfile(old_path):
```

```
        ext = os.path.splitext(file)[1]
```

```
        new_name = f"new_{count}{ext}"
```

```
        new_path = os.path.join(path, new_name)
```

```
        os.rename(old_path, new_path)
```

```
        print(f"Renamed: {file} → {new_name}")
```

```
        count += 1
```

```
def filter_files(path):
```

```
    ext = input("Enter extension (e.g., .txt): ")
```

```
    files = os.listdir(path)
```

```
    print("\nMatching files:")
```

```
    for file in files:
```

```
        if file.endswith(ext):
```

```
            print(file)
```

```
def main():
```

```
    path = input("Enter folder path: ")
```

```
    while True:
```

```
        print("\n1. List Files")
```

```
        print("2. Rename Files")
```

```
        print("3. Filter Files")
```

```
        print("4. Exit")
```

```
        choice = input("Enter choice: ")
```



```
if choice == '1':  
    list_files(path)  
  
elif choice == '2':  
    rename_files(path)  
  
elif choice == '3':  
    filter_files(path)  
  
elif choice == '4':  
    print("Exiting...")  
    break  
  
else:  
    print("Invalid choice!")
```

```
main()
```

PROJECT 1: Intelligent CLI-Based “Life Operating System” (LifeOS)

Objective

Build a **command-line productivity system** that acts like your personal OS to manage:

```
from utils import load_data, save_data
```

```
from datetime import datetime
```

```
FILE = "data/logs.json"
```

```
def log_energy():
```

```
    logs = load_data(FILE)
```

```
    energy = int(input("How do you feel (1-5): "))
```

```

entry = {
    "time": str(datetime.now()),
    "energy": energy
}

logs.append(entry)
save_data(FILE, logs)

return energy

from utils import load_data, save_data
from datetime import date

FILE = "data/habits.json"

def add_habit():
    habits = load_data(FILE)

    name = input("Habit name: ")

    habit = {
        "name": name,
        "streak": 0,
        "longest": 0,
        "history": []
    }

    habits.append(habit)
    save_data(FILE, habits)

def mark_done():
    habits = load_data(FILE)

```

```
today = str(date.today())
```

```
for h in habits:
```

```
    print(h["name"])
```

```
choice = input("Which habit done: ")
```

```
for h in habits:
```

```
    if h["name"] == choice:
```

```
        if today not in h["history"]:
```

```
            h["history"].append(today)
```

```
            h["streak"] += 1
```

```
            h["longest"] = max(h["longest"], h["streak"])
```

```
        else:
```

```
            print("Already marked today")
```

```
save_data(FILE, habits)
```

```
from user import load_user
```

```
from task import add_task, view_tasks, suggest_tasks
```

```
from habit import add_habit, mark_done
```

```
from energy import log_energy
```

```
from report import generate_report
```

```
def main():
```

```
    user = load_user()
```

```
    while True:
```

```
        print("\n1. Add Task")
```

```
        print("2. View Tasks")
```

```
        print("3. Add Habit")
```

```
        print("4. Mark Habit Done")
```

```
print("5. Log Energy")
print("6. Suggest Tasks")
print("7. Weekly Report")
print("8. Exit")
```

```
choice = input("Enter choice: ")
```

```
if choice == "1":
    add_task()
elif choice == "2":
    view_tasks()
elif choice == "3":
    add_habit()
elif choice == "4":
    mark_done()
elif choice == "5":
    energy = log_energy()
elif choice == "6":
    suggest_tasks(user["type"], energy)
elif choice == "7":
    generate_report()
elif choice == "8":
    break
```

```
if __name__ == "__main__":
    main()
```

```
from utils import load_data
```

```
def generate_report():
    tasks = load_data("data/tasks.json")
    habits = load_data("data/habits.json")
```

```
completed = [t for t in tasks if t["status"] == "Done"]
pending = [t for t in tasks if t["status"] == "Pending"]
```

```
print("\nReport:")
print("Completed Tasks:", len(completed))
print("Pending Tasks:", len(pending))
```

```
for h in habits:
    print(f"{h['name']} streak: {h['streak']}")
```

```
from utils import load_data, save_data
from datetime import datetime
```

```
FILE = "data/tasks.json"
```

```
def add_task():
```

```
    tasks = load_data(FILE)
```

```
    title = input("Task title: ")
    priority = int(input("Priority (1-5): "))
    energy = input("Energy (Low/Medium/High): ")
    deadline = input("Deadline (YYYY-MM-DD): ")
```

```
    task = {
        "title": title,
        "priority": priority,
        "energy": energy,
        "deadline": deadline,
        "status": "Pending"
    }
```

```
tasks.append(task)

save_data(FILE, tasks)
```

```
def view_tasks():

    tasks = load_data(FILE)

    for t in tasks:

        print(t)
```

```
def suggest_tasks(user_type, current_energy):

    tasks = load_data(FILE)
```

```
    print("\nSuggested Tasks:")
```

```
    for t in tasks:

        if t["status"] == "Pending":

            if current_energy <= 2 and t["energy"] == "Low":

                print(t["title"])

            elif current_energy >= 4 and t["priority"] >= 4:

                print(t["title"])
```

```
from utils import load_data, save_data
```

```
FILE = "data/user.json"
```

```
def create_user():

    name = input("Enter your name: ")

    hours = input("Daily available hours: ")

    ptype = input("Morning or Night person: ")
```

```
user = {

    "name": name,

    "hours": hours,
```

```

        "type": ptype
    }

    save_data(FILE, user)

    return user

def load_user():
    data = load_data(FILE)

    if not data:
        return create_user()

    return data

import json
import os

def load_data(file_path):
    if not os.path.exists(file_path):
        return []

    with open(file_path, 'r') as file:
        return json.load(file)

def save_data(file_path, data):
    with open(file_path, 'w') as file:
        json.dump(data, file, indent=4)

```

PROJECT 2: “Digital Investigation System” (Mini Forensics Tool)

Objective

Build a **CLI-based investigation system** that analyzes logs and detects suspicious activity.

Think like a **cyber investigator** 🕵️

```

def detect_suspicious(logs):
    suspicious = []

    failed_count = {}

```

```

night_users = set()
sensitive_users = set()

for log in logs:
    user = log["user"]

    # Count failed logins
    if log["action"] == "LOGIN" and log["status"] == "FAILED":
        failed_count[user] = failed_count.get(user, 0) + 1

    # Night login (store in set to avoid duplicates)
    if log["action"] == "LOGIN":
        hour = int(log["time"][:2])
        if 0 <= hour <= 5:
            night_users.add(user)

    # Sensitive file access
    if log["action"] == "FILE_ACCESS":
        if "secret" in log["file"]:
            sensitive_users.add(user)

# Add unique events
for user in night_users:
    suspicious.append((user, "Night Login"))

for user in sensitive_users:
    suspicious.append((user, "Sensitive File Access"))

for user, count in failed_count.items():
    if count >= 3:
        suspicious.append((user, "Multiple Failed Logins"))

```



```

    return suspicious

def parse_logs(file_path):
    logs = []

    with open(file_path, "r") as file:
        for line in file:
            parts = line.strip().split()

            log = {
                "date": parts[0],
                "time": parts[1].replace(":", ""),
                "user": parts[2],
                "action": parts[3],
                "status": parts[4] if len(parts) > 4 else "",
                "file": parts[4] if "FILE_ACCESS" in parts else ""
            }

            logs.append(log)

    return logs

def profile_users(logs):
    profiles = {}

    for log in logs:
        user = log["user"]

        if user not in profiles:
            profiles[user] = {
                "logins": 0,
                "failed": 0,

```

```
    "files": []  
}
```

```
if log["action"] == "LOGIN":  
    profiles[user]["logins"] += 1  
    if log["status"] == "FAILED":  
        profiles[user]["failed"] += 1
```

```
if log["action"] == "FILE_ACCESS":  
    profiles[user]["files"].append(log["file"])
```

```
return profiles
```

```
def search_by_user(logs, username):
```

```
    for log in logs:  
        if log["user"] == username:  
            print(log)
```

```
def generate_report(scores, suspicious):
```

```
    # open file with UTF-8 (so no encoding error)  
    with open("output.txt", "w", encoding="utf-8") as file:
```

```
        # Risk Scores Section
```

```
        file.write("=== Risk Scores ===\n")  
        for user, (score, level) in scores.items():  
            file.write(f"{user} -> {score} ({level})\n")
```

```
        # Suspicious Events Section
```

```
        file.write("\n=== Suspicious Events ===\n")  
        for event in suspicious:  
            file.write(f"{event[0]} -> {event[1]}\n")
```

```
print(" Report saved to output.txt")
```

```
def calculate_risk(profiles):  
    scores = {}  
  
    for user, data in profiles.items():  
        score = 0  
  
        score += data["failed"] * 2  
  
        for file in data["files"]:  
            if "secret" in file:  
                score += 5  
  
        if data["logins"] > 0:  
            score += 3 # assume night login exists  
  
        level = "LOW"  
        if score >= 8:  
            level = "HIGH"  
        elif score >= 4:  
            level = "MEDIUM"  
  
        scores[user] = (score, level)  
  
    return scores
```